What happens when a secret key leaks?

A four-minute live demo in three stages

A Exposed to the browser

- Component reads a NEXT_PUBLIC_ value
- Next.js inlines it into browser JavaScript at build time
- Find it in DevTools: header and .js file

The prefix makes a value public. It does not make a secret safe.

B Used on the server

- Browser → Next.js route → mock API
- Server adds the key; the browser never sees it
- Result returned to the browser is harmless

Hiding the key does not protect the route: limits, least privilege, authentication.

C Rotation and revocation

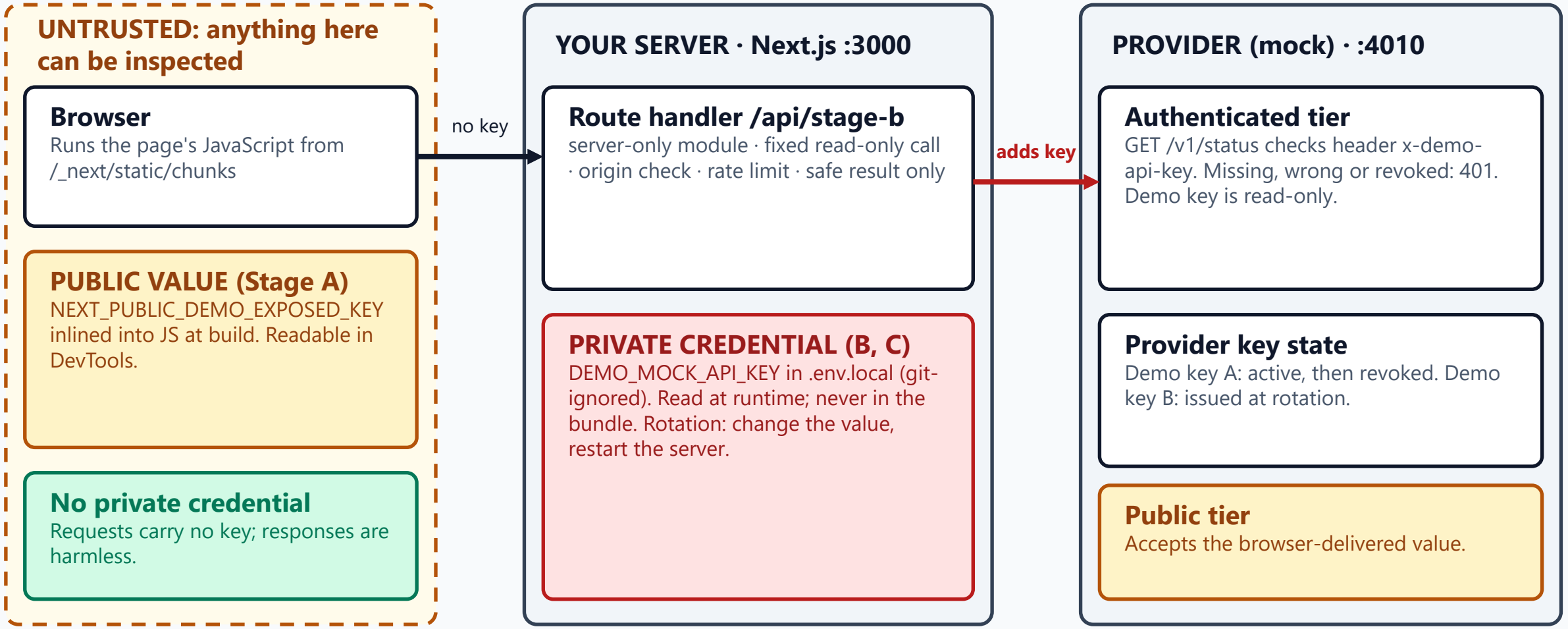
- Request works with Demo key A
- Provider issues Demo key B, revokes A: 401
- Update config, restart: works again

Removing a key from code does not invalidate it.

Educational demonstration · fake credentials · local mock API only

Where the public value and the private credential live

Browser → Next.js server route → mock API (all on 127.0.0.1)

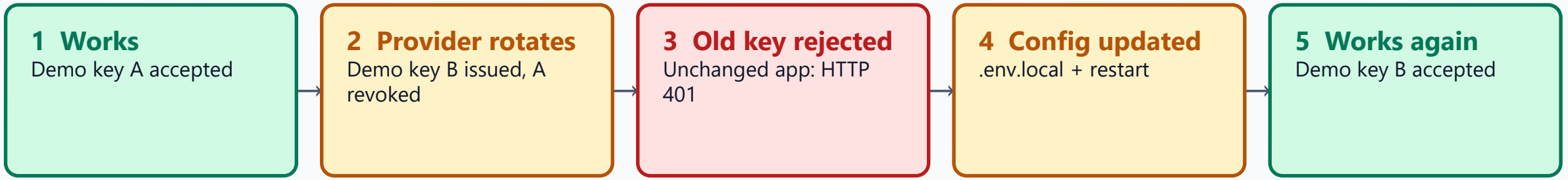


Stage A: the browser can also call the public tier directly with the exposed value

Educational demonstration · fake credentials · local mock API only

Rotation needs two updates

What Stage C showed, and what to take away: a hidden credential still needs protecting



- Browser-delivered values can be inspected.
- A server can use a credential without giving it to the browser.
- Removing a key from code does not invalidate it.
- Rotation must update both the provider and the application's configuration.
- A hidden credential still needs permissions, limits and protected endpoints.

Local simulation

Real providers differ. Planned rotation can overlap old and new keys briefly. An actively compromised key needs urgent revocation, which may interrupt service.

Sources: Next.js docs (environment variables, data security); GitHub “Remediating a leaked secret”; AWS Security Blog; OWASP Secrets Management. Links in SOURCES.md.